

11th Feb 2026 | DAGH01

→ Diff. types of GD

→ Vanilla GD

↳ compute the gradient using the entire training dataset in each update step

$$\theta_{t+1} = \theta_t - \eta \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} l(f(x^i), \theta_t), y^i)$$

$O(N \cdot d)$ $\leadsto$ time per update

$O(N)$ $\leadsto$ to store samples

Adv. $\rightarrow$ guaranteed convergence, low variance, smooth trajectory, deterministic updates

Disadv. $\rightarrow$ computationally expensive, slow updates, entire dataset in 1 go

→ SGD

↳ computes gradient using 1 random sample at a time.

$$E_i \sim U(1, N) [\nabla_{\theta} L^i(\theta)] = \nabla_{\theta} L(\theta) \quad \downarrow \text{unbiased estimator}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} l(f(x^i, \theta_t), y^i)$$

$O(d)$ $\rightarrow$ time per update

$O(1)$ $\rightarrow$ 1 sample at a time

Total updates / epoch $\Rightarrow N$

Adv. $\rightarrow$ fast updates, low memory, handles large datasets

Disadv. $\rightarrow$ high variance, noisy trajectory, may oscillate, req. careful η .

→ Mini-Batch GD

↳ computes grad. using a small batch of samples

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{b} \sum_{i \in I_t} \nabla_{\theta} l(f(x^i, \theta_t), y^i)$$

$O(b \cdot d)$ → time per update

$O(b)$ → stores mini-batch

$\lceil N/b \rceil \Rightarrow$ updates per epoch

Adv. → best of both, red. var., GPU acce., stable convergence, faster

Disadv. → req. tuning of batch-size, has some noise

	BGD	SGD	MBGD
samples per update	N	1	b
updates per epoch	1	N	$\lceil N/b \rceil$
$\text{Var}[\nabla_{\theta} L]$	0	σ^2	σ^2/b
itera ^{ns} per epoch	1	N	$\lceil N/b \rceil$

- more itera^{ns} per week lead to more frequent updates
- SGD has 1000 updates per epoch vs batch GD has 1 update per epoch. Mini batch helps them to strike a balance.